

```

import logging
import os
from telegram import (
    Update, InlineKeyboardButton, InlineKeyboardMarkup, ReplyKeyboardMarkup
)
from telegram.ext import (
    Application, CommandHandler, MessageHandler, CallbackQueryHandler,
    ConversationHandler, filters, ContextTypes
)

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# =====
# تنظیمات
# =====
BOT_TOKEN = os.environ.get("BOT_TOKEN", "YOUR_BOT_TOKEN")
ADMIN_ID = int(os.environ.get("ADMIN_ID", "0"))
CARD_NUMBER = os.environ.get("CARD_NUMBER", "6037-XXXX-XXXX-XXXX")
CARD_OWNER = os.environ.get("CARD_OWNER", "نام صاحب کارت")

# =====
# پلن های VPN
# =====
PLANS = {
    "plan_10": {"name": "۱۰ گیگ", "price": "۳۵,۰۰۰ تومان", "config": "PLAN_10_"},
    "plan_20": {"name": "۲۰ گیگ", "price": "۶۰,۰۰۰ تومان", "config": "PLAN_20_"},
    "plan_50": {"name": "۵۰ گیگ", "price": "۱۲۰,۰۰۰ تومان", "config": "PLAN_50_"},
    "plan_100": {"name": "۱۰۰ گیگ", "price": "۲۰۰,۰۰۰ تومان", "config": "PLAN_100_"},
    "plan_unlimited": {"name": "نامحدود ۳۰ روزه", "price": "۲۸۰,۰۰۰ تومان", "conf
}

# رو اینجا بذار VPN کانفیگ های واقعی
CONFIGS = {
    "PLAN_10_CONFIG": "vless://your-10gb-config-here",
    "PLAN_20_CONFIG": "vless://your-20gb-config-here",
    "PLAN_50_CONFIG": "vless://your-50gb-config-here",
    "PLAN_100_CONFIG": "vless://your-100gb-config-here",
    "PLAN_UNLIMITED_CONFIG": "vless://your-unlimited-config-here",
}

# state های ConversationHandler
WAIT_RECEIPT = 1

```

```

# ذخیره سفارش‌های در انتظار تایید: {message_id: order_info}
pending_orders: dict = {}

# =====
# /start
# =====
async def start(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    kb = [
        [InlineKeyboardButton("🛒 خرید VPN", callback_data="buy")],
        [InlineKeyboardButton("☎ پشتیبانی", callback_data="support")],
    ]
    await update.message.reply_text(
        "🔒 *فروشگاه VPN*\n\n"
        "خوش اومدی VPN سلام! به فروشگاه"
        "از منوی زیر گزینه مورد نظرت رو انتخاب کن",
        parse_mode="Markdown",
        reply_markup=InlineKeyboardMarkup(kb)
    )

# =====
# نمایش پلن‌ها
# =====
async def show_plans(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()

    kb = []
    for plan_id, plan in PLANS.items():
        kb.append([InlineKeyboardButton(
            f"📦 {plan['name']} - {plan['price']}",
            callback_data=f"select_{plan_id}"
        )])
    kb.append([InlineKeyboardButton("⬅️ بازگشت", callback_data="back_main")])

    await query.edit_message_text(
        "📦 *یکی از پلن‌ها رو انتخاب کن*\n\nپلن‌های موجود*",
        parse_mode="Markdown",
        reply_markup=InlineKeyboardMarkup(kb)
    )

# =====
# انتخاب پلن → نمایش قیمت و شماره کارت
# =====
async def select_plan(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()

```

```

plan_id = query.data.replace("select_", "")
plan = PLANS.get(plan_id)
if not plan:
    await query.edit_message_text("❌ پلن پیدا نشد.")
    return ConversationHandler.END

ctx.user_data["selected_plan"] = plan_id

kb = [[InlineKeyboardButton("✅ پرداخت کردم، رسید میفرستم", callback_data="pa
    [InlineKeyboardButton("⬅️ BACK بازگشت به پلن‌ها", callback_data="buy")]]

await query.edit_message_text(
    f"✅ *پلن انتخابی: {plan['name']}*\n\n"
    f"$ *مبلغ قابل پرداخت:*\n`{plan['price']}`\n\n"
    f"💳 *شماره کارت:*\n`{CARD_NUMBER}`\n\n"
    f"👤 *به نام:*\n`{CARD_OWNER}`\n\n"
    f"⚠️ *بعد از واریز، دکمه زیر رو بزن و *رسید* رو برام بفرست",
    parse_mode="Markdown",
    reply_markup=InlineKeyboardMarkup(kb)
)
return WAIT_RECEIPT

# =====
#   کاربر گفت پرداخت کردم → منتظر رسید
#   =====
async def ask_receipt(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    await query.edit_message_text(
        f"🔔 *رسید پرداخت رو بفرست*\n\n"
        f"\nمی‌تونی عکس یا اسکرین‌شات رسید رو بفرستی"
        f"✅ *بعد از تایید ادمین، کانفیگ برات ارسال میشه",
        parse_mode="Markdown"
    )
    return WAIT_RECEIPT

# =====
#   دریافت رسید از کاربر
#   =====
async def receive_receipt(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    user = update.effective_user
    plan_id = ctx.user_data.get("selected_plan", "نامشخص")
    plan = PLANS.get(plan_id, {})

    # پیام تایید به کاربر
    await update.message.reply_text(
        f"🕒 *رسیدت دریافت شد*\n\n"

```

```

"\n".برات ارسال میشه VPN بعد از بررسی توسط ادمین، کانفیگ
", معمولاً کمتر از ۳۰ دقیقه طول میکشه"
parse_mode="Markdown"
)

# ارسال رسید به ادمین
caption = (
    f"🔔 *سفارش جدید*\n\n"
    f"👤 کاربر: {user.full_name}\n"
    f"🆔 آیدی: `{user.id}`\n"
    f"📦 پلن: {plan.get('name', plan_id)}\n"
    f"💰 مبلغ: {plan.get('price', '-')}\n\n"
    f"👉 برای تایید و ارسال کانفیگ روی دکمه بزن"
)

kb = [[
    InlineKeyboardButton("✅ تایید و ارسال کانفیگ", callback_data=f"approve_{
    InlineKeyboardButton("❌ رد کردن", callback_data=f"reject_{user.id}"),
]]

# رسید عکس یا متن؟
if update.message.photo:
    msg = await ctx.bot.send_photo(
        ADMIN_ID,
        photo=update.message.photo[-1].file_id,
        caption=caption,
        parse_mode="Markdown",
        reply_markup=InlineKeyboardMarkup(kb)
    )
else:
    text = update.message.text or update.message.caption or "بدون متن"
    msg = await ctx.bot.send_message(
        ADMIN_ID,
        text=f"{caption}\n\n📄 متن رسید:\n{text}",
        parse_mode="Markdown",
        reply_markup=InlineKeyboardMarkup(kb)
    )

pending_orders[msg.message_id] = {"user_id": user.id, "plan_id": plan_id}
return ConversationHandler.END

# =====
# ادمین: تایید سفارش
# =====
async def approve_order(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()

```

```

if update.effective_user.id != ADMIN_ID:
    await query.answer("❌ دسترسی ندارید.", show_alert=True)
    return

parts = query.data.split("_") # approve_USERID_PLANID
user_id = int(parts[1])
plan_id = parts[2]
plan = PLANS.get(plan_id, {})
config_key = plan.get("config", "")
config = CONFIGS.get(config_key, "❌ کانفیگ تنظیم نشده")

# ارسال کانفیگ به کاربر
await ctx.bot.send_message(
    user_id,
    f"📦 سفارشت باید شد *\n\n"
    f"📦 پلن: *{plan.get('name', plan_id)}*\n\n"
    f"🔑 *VPN کانفیگ:*\n`{config}`\n\n"
    f"📱 وارد کن Hiddify یا V2rayNG این لینک رو کپی کن و توی اپ\n"
    f"🚀!موفق باشی",
    parse_mode="Markdown"
)

await query.edit_message_reply_markup(reply_markup=None)
await ctx.bot.send_message(ADMIN_ID, f"✅ {user_id} کانفیگ برای کاربر")

# =====
# ادمین: رد سفارش
# =====
async def reject_order(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()

    if update.effective_user.id != ADMIN_ID:
        await query.answer("❌ دسترسی ندارید.", show_alert=True)
        return

    user_id = int(query.data.split("_")[1])

    await ctx.bot.send_message(
        user_id,
        "❌ متأسفانه رسید شما باید نشد *\n\n"
        "لطفاً دوباره رسید معتبر ارسال کنید یا با پشتیبانی تماس بگیرید",
        parse_mode="Markdown"
    )

    await query.edit_message_reply_markup(reply_markup=None)

```

```

        await ctx.bot.send_message(ADMIN_ID, f"رد شد {user_id} سفارش کاربر ✖")

# =====
# پشتیبانی
# =====
async def support(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    await query.edit_message_text(
        "📞 *پشتیبانی*\n\n"
        ".برای ارتباط با پشتیبانی پیام بفرست"
        ", ساعت پاسخگویی: ۹ صبح تا ۱۲ شب",
        parse_mode="Markdown",
        reply_markup=InlineKeyboardMarkup([[
            InlineKeyboardButton("⬅️ بازگشت", callback_data="back_main")
        ]])
    )

# =====
# بازگشت به منوی اصلی
# =====
async def back_main(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    kb = [
        [InlineKeyboardButton("🛒 خرید VPN", callback_data="buy")],
        [InlineKeyboardButton("📞 پشتیبانی", callback_data="support")],
    ]
    await query.edit_message_text(
        "🔒 *VPN فروشگاه*\n\n"
        ", از منوی زیر گزینه مورد نظرت رو انتخاب کن",
        parse_mode="Markdown",
        reply_markup=InlineKeyboardMarkup(kb)
    )

# =====
# اجرای اصلی
# =====
def main():
    app = Application.builder().token(BOT_TOKEN).build()

    conv = ConversationHandler(
        entry_points=[CallbackQueryHandler(select_plan, pattern="^select_")],
        states={
            WAIT_RECEIPT: [
                CallbackQueryHandler(ask_receipt, pattern="^paid$"),
                MessageHandler(filters.PHOTO | filters.TEXT & ~filters.COMMAND, r

```

```

        ]
    },
    fallbacks=[CommandHandler("start", start)],
    per_message=False,
)

app.add_handler(CommandHandler("start", start))
app.add_handler(CallbackQueryHandler(show_plans, pattern="^buy$"))
app.add_handler(CallbackQueryHandler(support, pattern="^support$"))
app.add_handler(CallbackQueryHandler(back_main, pattern="^back_main$"))
app.add_handler(CallbackQueryHandler(approve_order, pattern="^approve_"))
app.add_handler(CallbackQueryHandler(reject_order, pattern="^reject_"))
app.add_handler(conv)

logger.info("Bot started...")
app.run_polling()

if __name__ == "__main__":
    main()

```